DEEP NEURAL NETWORKS

Adrian Jackson

a.jackson@epcc.ed.ac.uk

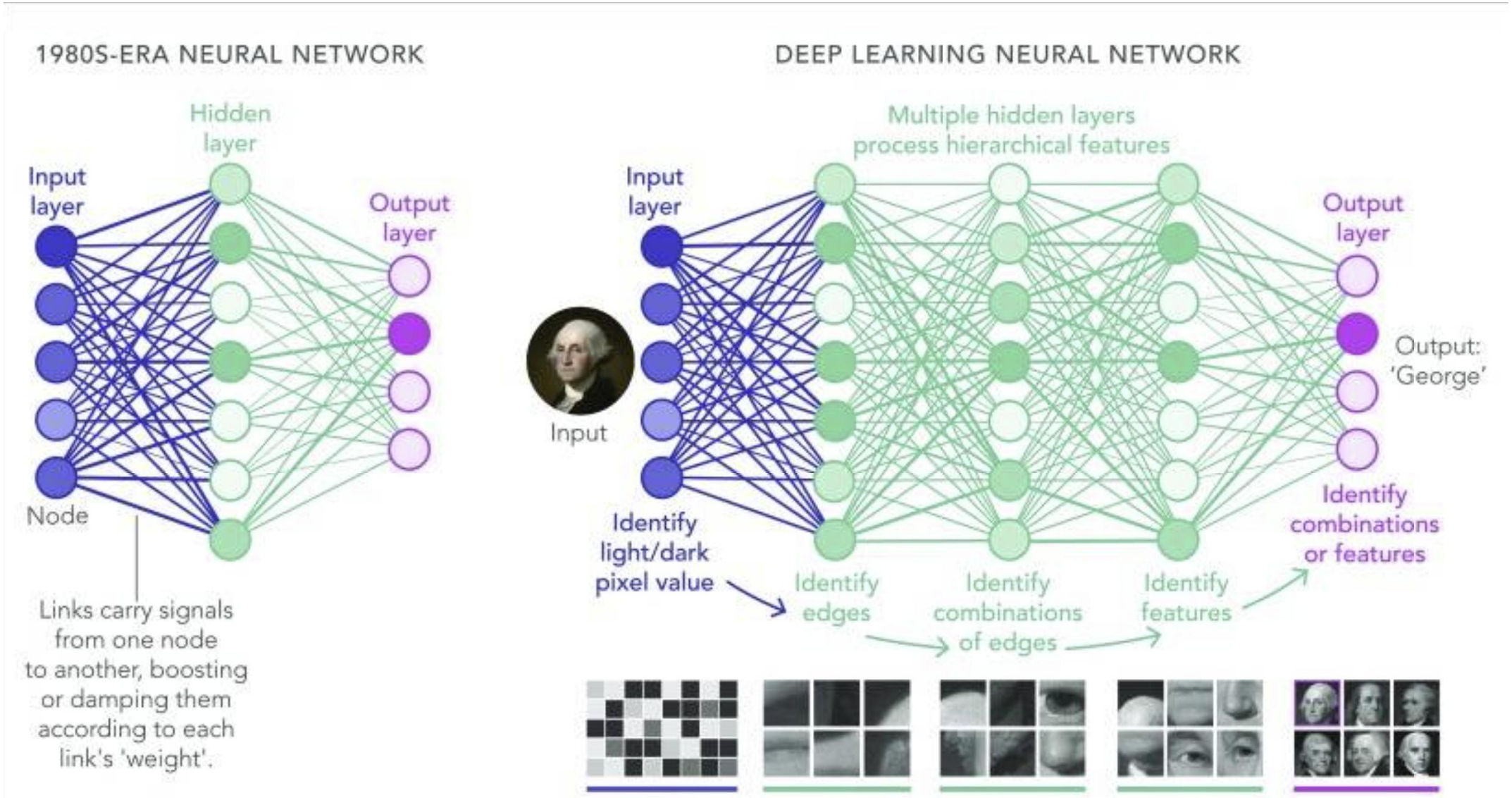
THE UNIVERSITY of EDINBURGH

epcc

Deep Neural Networks (DNNs)

- DNNs subset of machine learning models
  - inspired by the structure and function of the human brain's neural networks
  - Multiple interconnected layers of functions (neurons)
- Good at capturing complex patterns and representations from data
  - Suitable for tasks like image recognition, natural language understanding, and speech recognition
- Components
  - **Input Layer**: Receives input data, which can be images, text, or numerical features
  - **Hidden Layers**: Multiple layers of neurons that can perform the feature extraction and data transformation
  - **Output Layer**: Produces predictions or classifications based on the learned features
- Activation Functions
  - Allows switching on/off individual components
  - Adds non-linear functionality to the network
  - Enables complex relationships
- DNNs functionality needs to be trained
  - Update the contribution of each part of the input to the output, via the hidden layers
  - Weights updating and loss function key to this

DNNs



https://www.ncbi.nlm.nih.gov/pmc/articles/PMC6347705/

Neurons

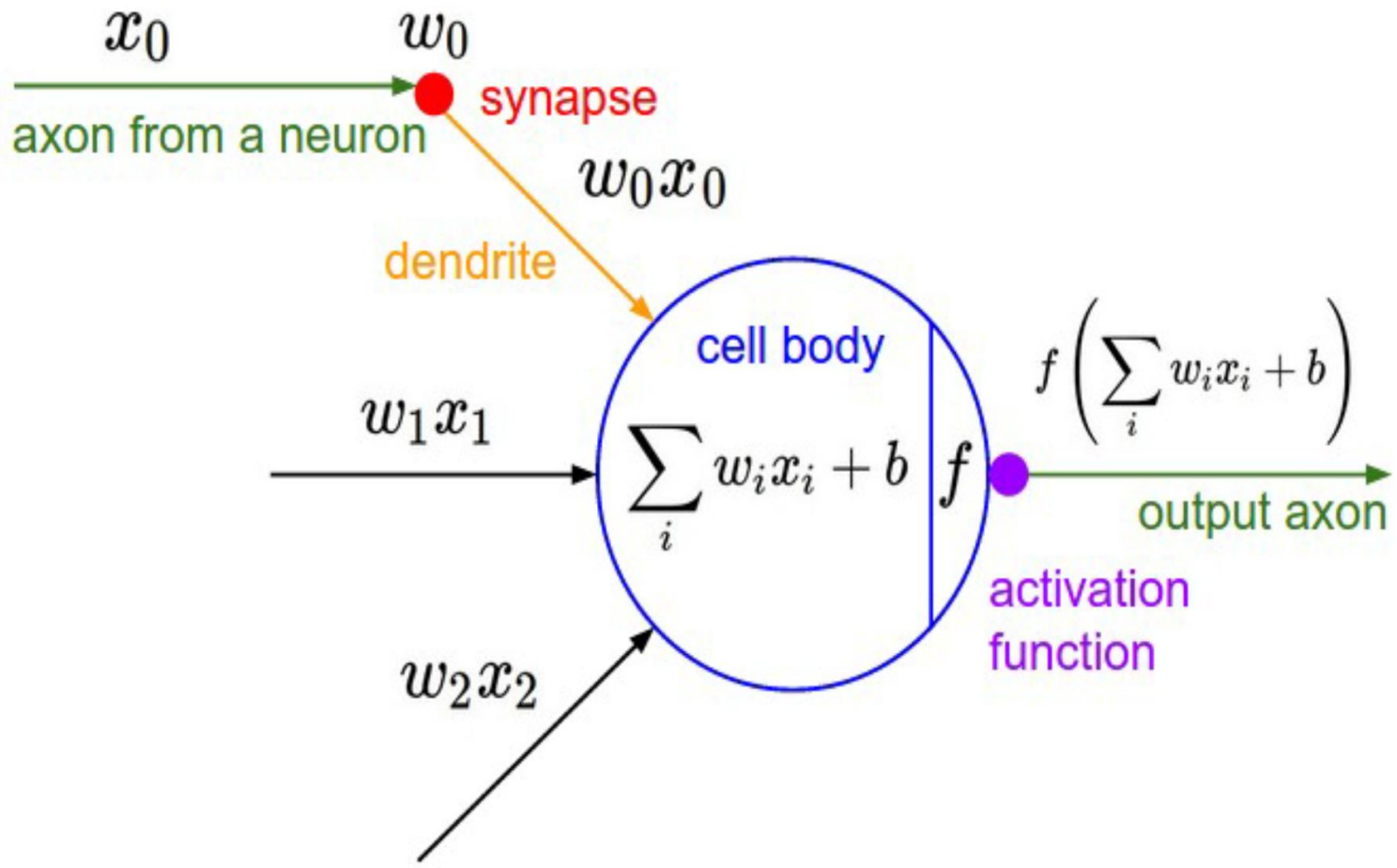
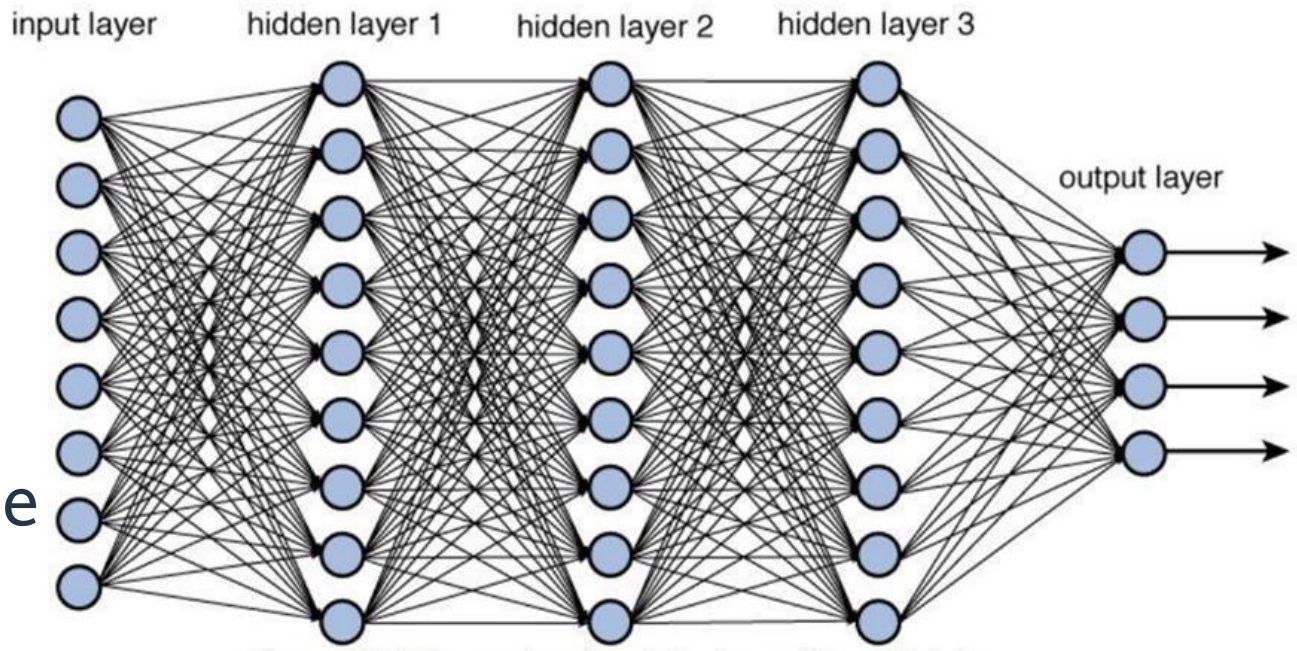
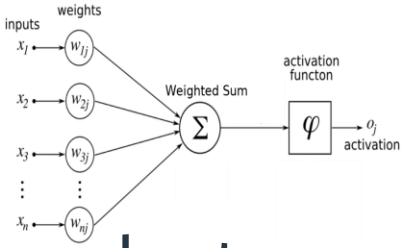


Image Source: Stanford

Networks of neurons

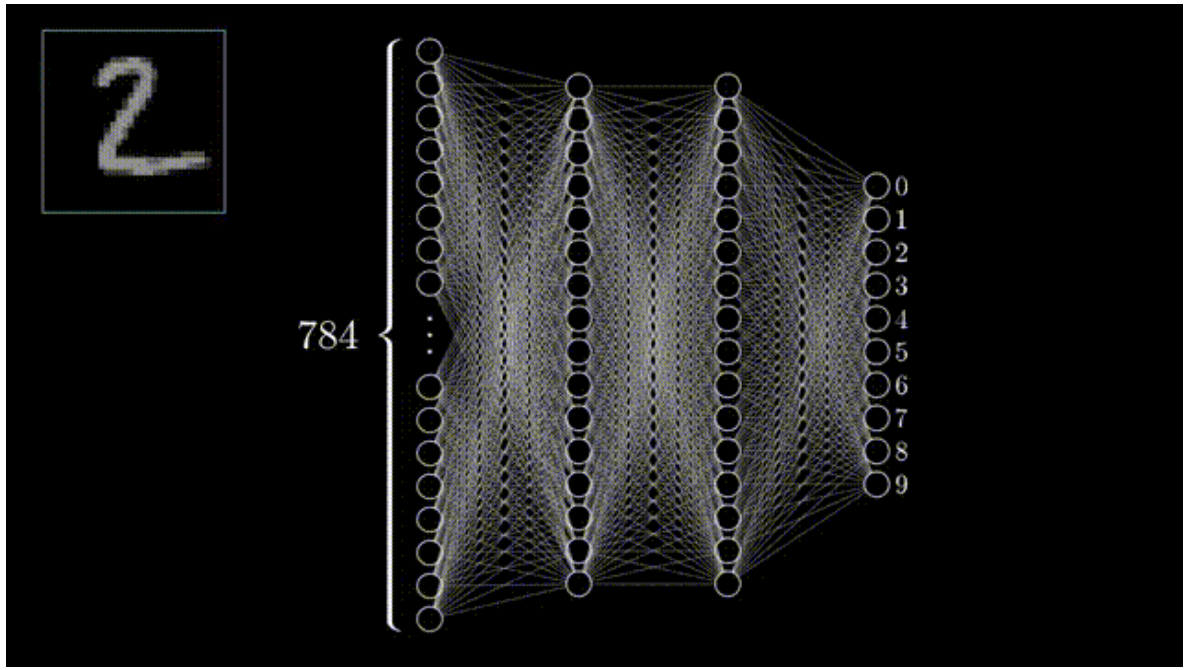


- DNN have multiple layers
  - Hidden layer capture data features
- Each layer can have variable numbers of neurons
- Weights on each connection



- Also have a bias connection per neuron/layer
- Specific configuration/construction of layers dependent on network type and problem domain
  - Number/size of layer impacts computational cost, memory requirements, and generalisability

Forward Propagation (inference)

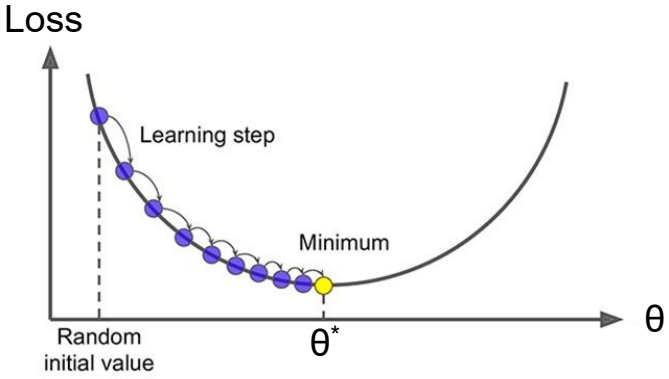


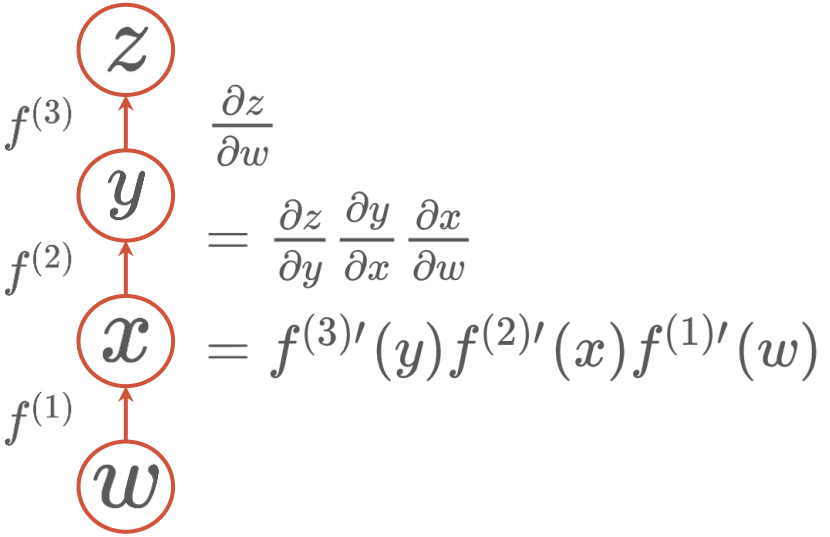
Animation adapted from: youtube.com/watch?v=aircAruvnKk&feature

- Take input and pass through
  - Generate prediction/answer
- General matrix-matrix form
  - Sum of functions of weight x input
- Each neuron has
  - k weights and inputs
  - Formula is $f(\sum_{i=1}^{n}(w_i x_i) + b)$ independent neuron (node) updates
  - Can be mapped to matrix – matrix calculations
  - i.e. $Z_1 = W_1 X + B_1$
- Also need to apply activation calculation, i.e.
  - $A_1 = g_1(Z_1)$
- Continue for next layers, i.e. $Z_2 = W_2 A_1 + B_2; A_2 = g_2(Z_2)$

Backward Propagation (training)

- Update weights and biases using losses
  - Loss function defines “distance” from correct answer
  - Iteratively update to get close to correct
- Chain rule used to derive formulation that relates the loss between output and ground truth to the component neuron weights and bias variables
  - Can then update them to reduce the loss value and then repeat the process
- Generalised training uses gradient descent
  - Loss function after all elements used for training (full epoch)
  - Computational expensive for large networks/datasets





Loss functions

- Range of loss functions depending on what type of training/network being used:
  - Mean Squared Error (MSE) Loss
    - L2 (quadratic) loss
    - Commonly used for regression tasks
    - Calculates the average squared difference between predicted and actual values
    - Minimize the variance between predictions and true targets
  - Binary Cross-Entropy Loss (Logistic):
    - Used for binary classification problems (target variable 0 or 1)
    - Measures the difference between predicted probabilities and true labels
  - Categorical Cross-Entropy Loss (Softmax):
    - Multi-classification tasks (more than 2 classes)
    - Uses binary encoding of classes to compare (i.e. [0,0,1])
    - Aims accurate probability distributions over all classes

Training approaches

```
while (loss too high):
  for each epoch: // a pass through items in training dataset
     grad = 0
     for each item x_i in training set:
        grad += evaluate_loss_gradient(f, params, loss_func, x_i)
     params += -grad * learning_rate;
```

- Other alternatives exist for training
  - Gradient descent requires full dataset pass per iteration
  - Accurate loss function but expensive

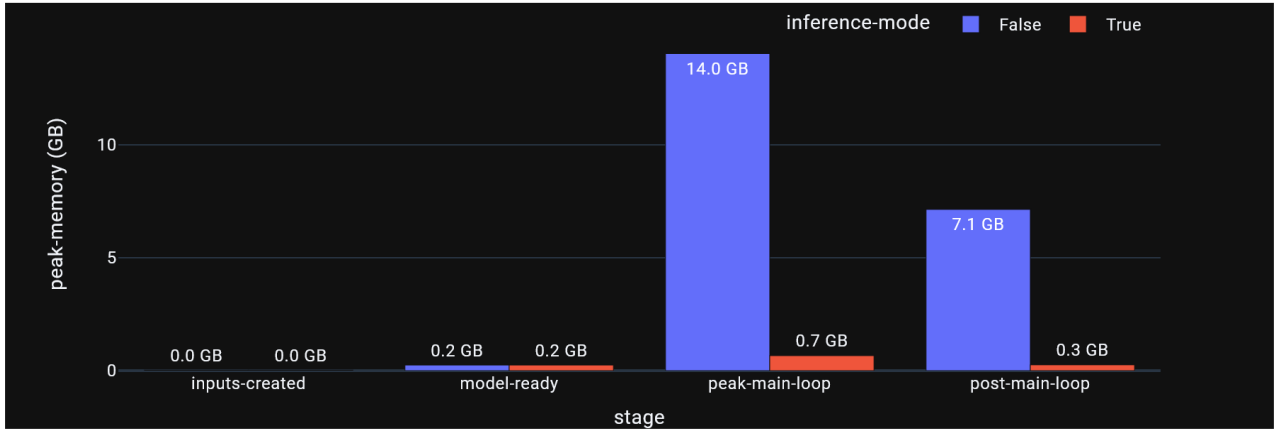
```
while (loss too high):
  for each epoch:
     for all mini batches in training set:
       grad = 0;
       for each item x_i in minibatch:
          grad += evaluate_loss_gradient(f, params, loss_func, x_i)
       params += -grad * learning_rate;
```

- Stochastic gradient descent
  - Variant where subset of data is used for each update cycle (epoch)
  - Pure SGD would update weights after every data point
  - Individual inputs processed separately
  - Weight updates/loss minimisation for each
  - Individual solutions
  - Batched-SGD does this after n data points (smooths the loss graph) (mini-batch)
- More complex optimisers also available
  - ADAM, RMSProp, etc...
  - More computational intensive but may have better convergence properties

DNN forward and backward pass (training)

- Classic wavefront computation between layers
  - Both forward and backwards passes require layer n to be done before n+1 can be done
  - Restricts the level of parallelism
- Convergence required for optimiser
  - Zero loss
  - Fixed iteration space
- Hyperparameters for tuning the network
  - Learning rate
  - Batch size
  - Number of training iterations
  - etc...

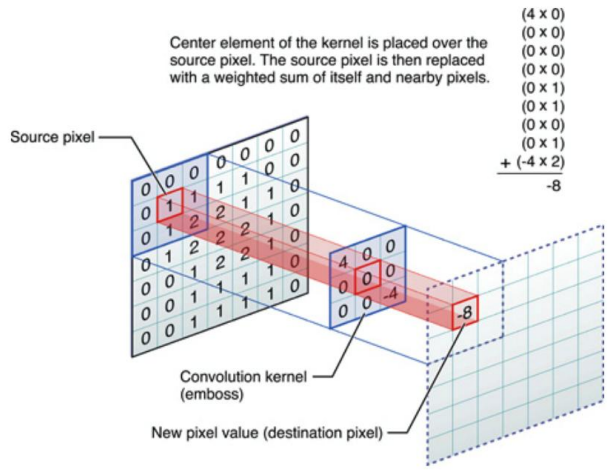
Memory usage



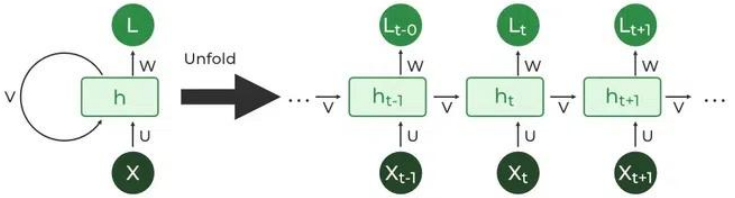
- Inference
  - Only need to retain transient output
  - i.e. only need layer n-1 output for layer n
  - Need full weights
- Training
  - Need all layer outputs for gradient calculation in back propogration
  - Expensive memory requirements
  - Necessitates using multiple GPUs for large networks

Types of DNN

- Range of different types of deep networks, focussed on different tasks
- Feedforward Neural Networks (FNNs)
  - Convolutional Neural Network (CNN)
    - Specialized for processing grid-like data, e.g., images
    - Employs convolutional layers for feature extraction and spatial hierarchies
    - Image and video analysis, object detection, facial recognition, etc
    - Spatial data where local patterns are significant
- Recurrent Neural Network (RNN)
  - Process sequences of data with feedback loops
  - Allows some state to be maintained during feed forward
  - Natural language processing, speech recognition, time series prediction
  - Tasks where temporal dependencies matter



Understanding Convolutional Neural Networks for NLP. WildML. 7th November 2015



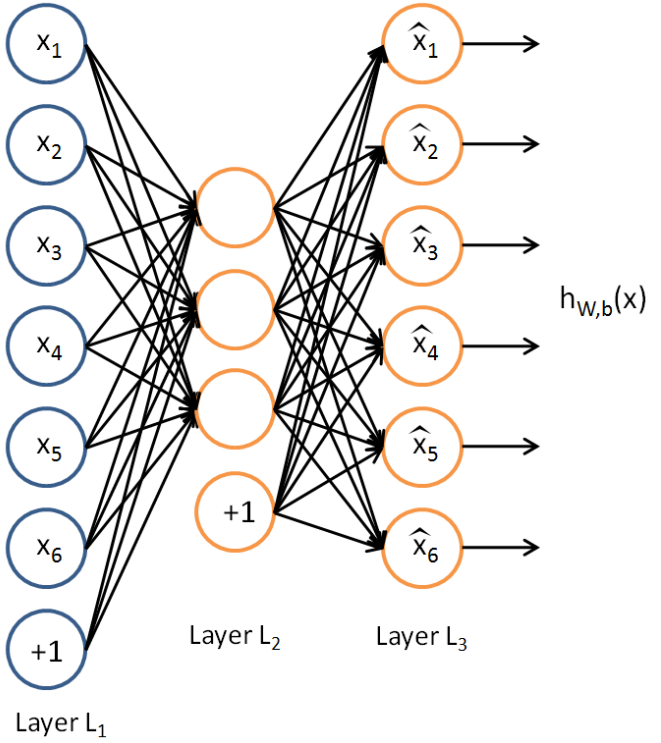
https://www.geeksforgeeks.org/introduction-to-recurrent-neural-network/

Types of DNN

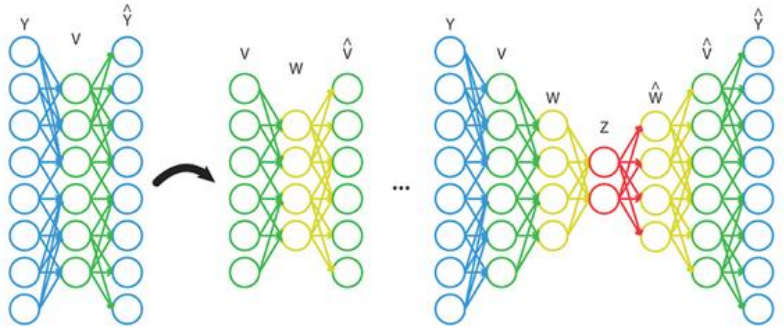
- Long Short-Term Memory (LSTM) Network
  - A type of RNN with specialized cells to handle long-range dependencies.
  - Addresses vanishing gradient problem
  - Language modeling, machine translation, sentiment analysis
- Gated Recurrent Unit (GRU) Network
  - Like LSTM but with simplified gating mechanisms
  - Balances performance and complexity in sequence modeling
  - Speech synthesis, recommendation systems, chatbots

Types of DNN

- Autoencoder
  - Neural network used for dimensionality reduction and feature learning
  - Comprises an encoder to map data to a lower-dimensional representation and a decoder for reconstruction
  - Layers trained separately with input == output
  - Data compression, image denoising, anomaly detection.
  - Feature learning for subsequent tasks
- Can also use multiple DNN together, i.e. Generative Adversarial Network (GAN)
  - Generator network to create data
  - Discriminator network to try and identify genuine data
  - Generator aims to produce data like the real data
  - Discriminator distinguishes between real and fake data
  - Loss functions to optimise real data production and identification

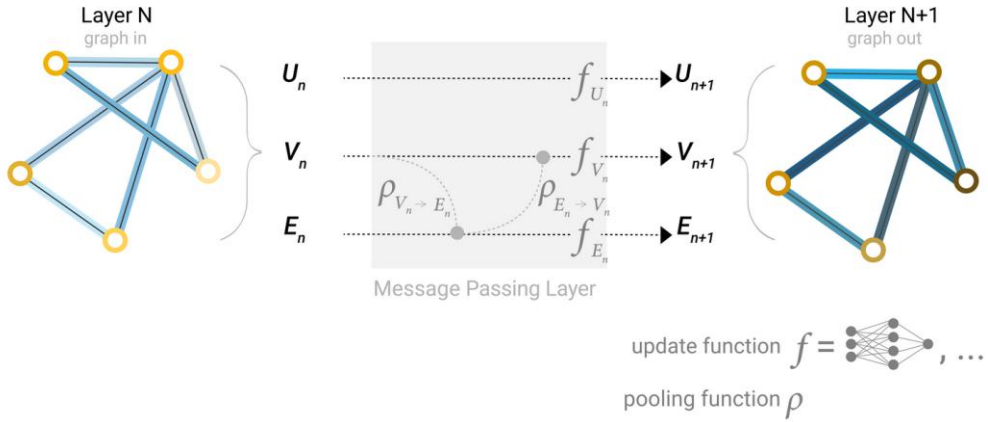


http://ufldl.stanford.edu/tutorial/unsupervised/Autoencoders/



Berniker M, Kording KP. Deep networks for motor control functions. *Frontiers in Computational Neuroscience*. 2015;9(32). doi:10.3389/fncom.2015.00032.

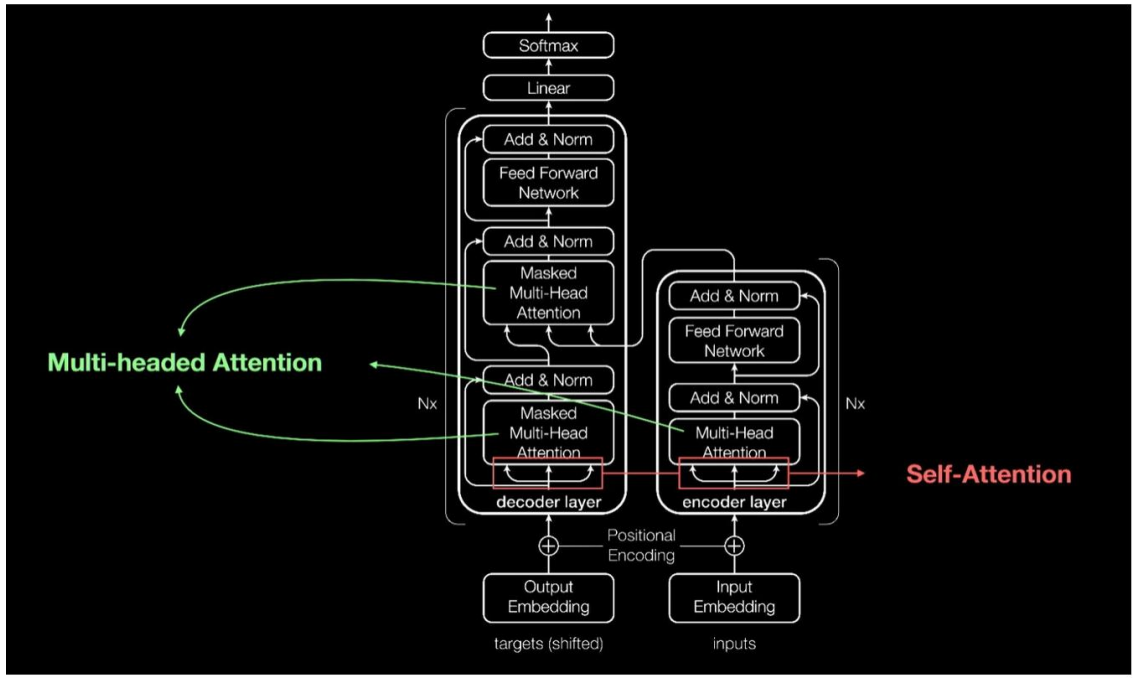
Graph Neural Networks (GNN)



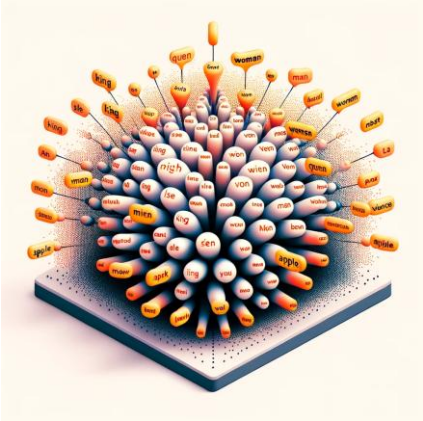
https://distill.pub/2021/gnn-intro/

- Different approaches to GNN
  - Graph: Whole structure categorisation
  - Node: Classification of elements
  - Edge: Relationship between elements
  - Collapsing graphs to matrices for computation so care about adjacency
- Typically have a multi-level perceptron (MLP) for each aspect of the graph (nodes, edges, etc...)
  - Plus pooling layers to couple these together (nodes influence edges, etc...)
- Nearest neighbour message passing to maintain locality
  - Variable form of convolution (no fixed kernels)

Transformers

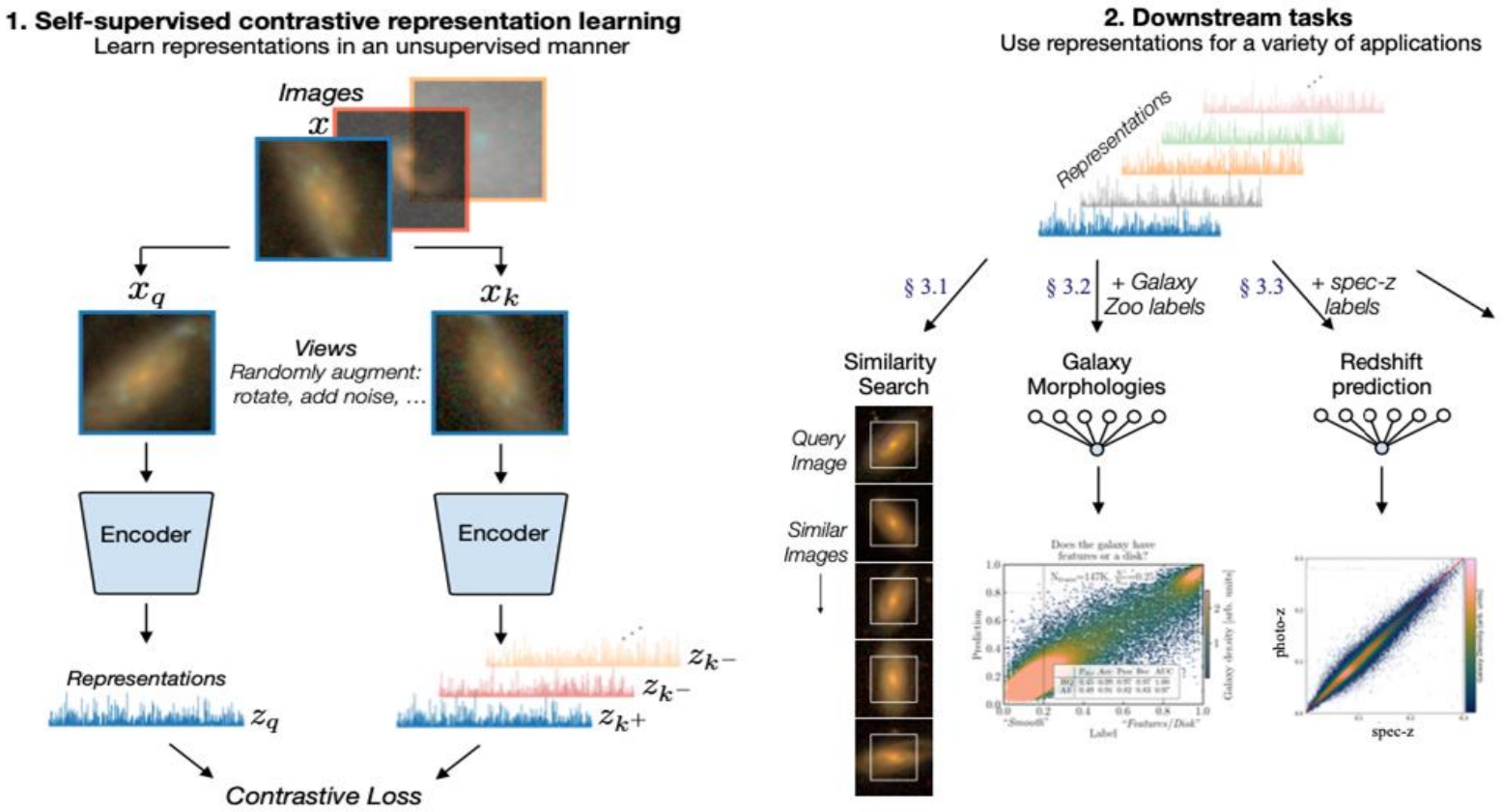


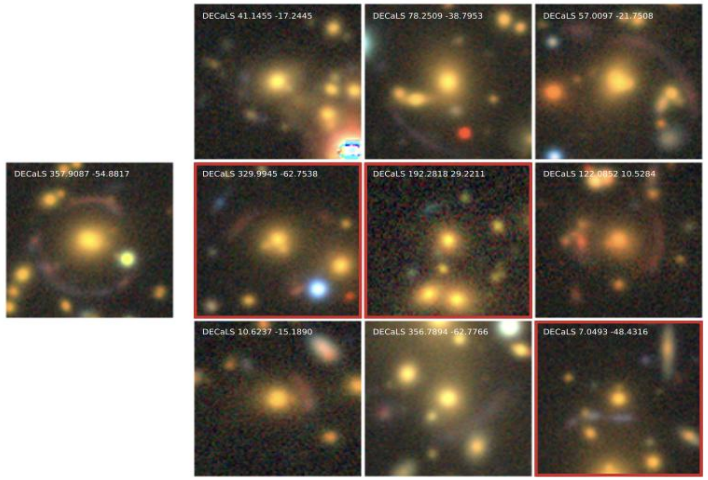
https://blogs.nvidia.com/blog/what-is-a-transformer-model/



- LLM built on transformer models
  - The feed forward network in the encoder and decoder is still a standard DNN (if often very large)
  - GPT-3 has something like 95 layers, 175 billion parameters (proportional to neurons)
  - Attention calculations are small matrix operations on word placement
    - Attention scores
- Word embeddings key part of transformers
  - Mapping words into high dimensional space
  - Embedding is learned as the model is trained

Example use: Identifying galaxy features





Initial approach: Hayat et. al. (2020) arXiv:2012.13083
Strong-lens analysis: Stein et. al. (2021) arXiv:2110.00023

DNNs

- Key to DNNs usefulness are
  - Growth in computing to enable large linear algebra operations to be undertaken quickly
  - Depth and breadth of the network has improved performance
- Still many challenges to functionality
  - Over fitting
  - Exploding or vanishing gradients
  - Reliability/Generalisability
  - Network design
  - Data availability
- HPC challenges are common
  - Parallelisation
  - Memory capacity
  - Communication
  - Load balance
- Try out some network features for yourselves
  - https://playground.tensorflow.org